certora

Security Assessment
Fuzzing and Invariant Testing
Final Report

Veda Boring Vault

January 2026

Prepared for Veda Team

certora

Table of content

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Veda Boring Vault	https://github.com/Veda-Labs/boring-vault	acad413	Solidity
Fuzzing	https://github.com/Certora/Veda-Fuzzing	acad413	Foundry, Medusa

Project Overview

This document describes the extensive fuzzing and invariant testing of **Veda** using Foundry. The first manual audit was undertaken from **August 28, 2025,** to **September 14, 2025**, formal verification took place from **September 18, 2025** to **October 24, 2025,** followed by a secondary review from **December 25, 2025** to **January 5, 2026**. The current engagement took place from **January 7, 2026** to **February 4, 2026**.

During this period, Certora's Security Research and Formal Verification team conducted comprehensive testing of the following Solidity contracts:

- `src/base/BoringVault.sol`
- `src/base/Roles/TellerWithMultiAssetSupport.sol`
- `src/base/Roles/TellerWithYieldStreaming.sol`
- `src/base/Roles/AccountantWithRateProvider.sol`
- `src/base/Roles/AccountantWithYieldStreaming.sol`

Protocol Overview

Veda is a cross-chain, multi-asset vault protocol designed to manage deposits, yield, and liquidity modularly, leaning on its trusted roles for executing sensitive operations. At its core, it combines its `BoringVault` with extensible user-facing `Teller` contracts, yield integrations with external protocols, cross-chain messaging powered by LayerZero and dynamic exchange rates, following the yield generation.

Invariant testing

Testing Methodology

We performed verification of the **Veda** protocol using the Foundry framework and Medusa, compiling all pre-existing invariants, rewritten from the Certora Verification Language (CVL) into Solidity, and **Veda**'s implementation source code written in Solidity.

The suite uses a standard Handler approach, which is meant to mold the way the source contracts are called, to closely keep track of the state before and after each call, define allowed actors, tokens and mock contracts and the invariants are run on this tracked state. However handlers must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter).

Invariants: An invariant is a universal property of a system that must hold true at all times, specifically after the deployment (base case) and after the execution of any transaction (inductive step). In fuzzing, invariants are expressed as boolean expressions or "properties" that the tool attempts to falsify by exploring the contract's state space; if the fuzzer finds a sequence of inputs where the expression evaluates to false, a vulnerability or logic error has been identified.

Handlers: A handler is a wrapper contract that acts as a middleware between the fuzzer and the target protocol to define and constrain the "action space" of the test. Instead of allowing the fuzzer to call low-level functions with raw, unconstrained data—which often leads to useless reverts—a handler implements high-level logic to set up valid state, manage multiple actors, and prune invalid inputs. This ensures the fuzzer spends its computational budget exploring deep, meaningful execution paths rather than hitting shallow input validation checks.

Verification Notations

Fuzz Verified	The invariant held across all runs with no violations. The property was tested extensively with randomized inputs across the bounded state space.
Violated	A counter-example sequence exists that violates the invariant.
Conditionally Verified	The invariant holds under specific preconditions (e.g., skipped when paused, skipped after fee claims). These conditions represent expected transient states where the property temporarily doesn't apply.

General Assumptions and Simplifications

- We address 2 combinations of these contracts which we named:
  - `RP System`: TellerWithMultiAssetSupport + AccountantWithRateProviders + BoringVault
  - `YS System`: TellerWithYieldStreaming + AccountantWithYieldStreaming + BoringVault
- We assume that the contracts are correctly linked:
  - Teller.vault == BoringVault
  - BoringVault.hook == Teller
  - Teller.accountant == Accountant
  - Accountant.vault == BoringVault
- Architectural Simplifications:
  - There could be multiple Tellers managing the same Vault but we take into account only 1 Teller per vault for simplicity
  - We do not test the BoringVault.manage() function (strategist operations) - we assume the vault's asset balance changes only via deposits, withdrawals, and yield vesting
- We do not test cross-vault interactions or migrations between systems

- We use mock contracts for assets, WETH, and the rate providers of the alternative assets. By doing this we explicitly test any number of alternative assets via plugging new asset-provider pairs
- Mock rate providers allow arbitrary rate manipulation to test edge cases
- All mock tokens have configurable decimals (6, 8, 11, 18) to test decimal handling
- Role and Permission Assumptions:
  - The owner, strategist, solver, and payoutAddress are distinct addresses with appropriate roles pre-configured
  - Role assignments are immutable during fuzzing - we do not test role changes
  - Authorization (via RolesAuthority) is assumed to be correctly configured at setup
- Yield Streaming Specific:
  - Yield is simulated by minting base assets to the vault proportionally to vesting progress (syncVestedAssets)
  - We do not simulate external yield sources - all yield comes from explicit vestYield calls
  - Time advancement via vm.warp triggers vesting calculations
- Actor Model:
  - A fixed pool of actors (users) interact with the system
  - One dedicated deniedUser for testing deny list functionality
  - One dedicated solver for bulk operations
  - Actors are pre-funded with sufficient tokens for all operations
- Economic Assumptions:
  - Exchange rates are bounded to realistic ranges (0.001 to 1000 tokens per share)
  - Deposit/withdrawal amounts are bounded (1 wei to 100,000 tokens and also percentages of the total)
  - Fee percentages are bounded (platform fee ≤ 20%, performance fee ≤ 50%)
  - Vesting durations are bounded (1 hour to 90 days)
- Known Limitations:
  - We do not test reentrancy scenarios (all mocks are non-reentrant)
  - We do not test gas griefing
  - We do not test permit/signature-based flows (depositWithPermit)
- Flash loan interactions are not extensively tested, they are simulated via excessively large deposits being allowed by the fuzzer

- Invariant Checking:
  - Invariants are checked after each handler call completes
  - Pre/post state snapshots capture only fields relevant to invariants (minimal to avoid stack-too-deep)
  - Solvency invariants skip checking immediately after fee claims or rate provider changes (transient states)

Detailed invariants

Solvency Assumptions:

Single-asset solvency formula:
`asset.balanceOf(Vault) * ONE_SHARE >= Vault.totalSupply() * accountant.getRateInQuoteSafe(asset)`

Multi-asset solvency formula:
`Σ (asset[i].balanceOf(Vault) * ONE_SHARE / getRateInQuoteSafe(asset[i])) >= totalSupply`

Module Properties

We verify the two vaults with their own pairs of contracts, where the YS system is a direct inheritor of the RP system, leading to some invariants duplicating and covering both systems

Group 1: Accountant Properties (Both Systems)	
Status: Conditionally Verified	

Invariant Name	Status	Description
accountantDoesntHoldTokens	Verified	*Accountant never holds tokens; payout address ≠ accountant*
accountantPaused_valuesFrozen	Verified	*When paused, fees only change via resetHighwaterMark*
feesCanOnlyDecreaseViaClaimFees	Verified	*feesOwedInBase decreases only via claimFees*
highwaterMarkNeverDecreases	Verified	*`HWM never decreases except via resetHighwaterMark`*

lastUpdateTimestampNeverDecreases	Verified	*Timestamp monotonically increases*
allowedExchangeRateChangeBounds	Verified	*upper >= 10000, lower <= 10000*
exchangeRateLEhighwaterMark	Required a manual handler workaround	*When not paused: rate <= HWM (checked after successful updateExchangeRate)*

Group 2: YS-Specific Properties

Status: Conditionally Verified

Rule Name	Status	Description
cumulativeSupplyBounded	Verified	*cumulativeSupplyLast <= cumulativeSupply*
exchangeRateEqlastSharePrice	Required a manual handler workaround	*After sync ops: exchangeRate == lastSharePrice*
sharePriceBoundedUpper	Verified	*rate * totalSupply / ONE_SHARE <= totalAssets + 1*
sharePriceBoundedLower	Verified	*rate * totalSupply / ONE_SHARE >= totalAssets (with tolerance)*
sharePriceMoreThanOne	Verified	*Rate > 0 when assets exist*
totalAssetsCovered	Required a manual handler workaround	*totalAssets <= vaultBalance (skipped after fee claims)*

Group 3: Teller & Vault Integrity (Both Systems)

Status: Verified

Rule Name	Status	Description
integrityOfDeposit	Verified	*Deposit with assets > 0 mints shares > 0*
integrityOfWithdraw	Verified	*Withdraw with shares > dust produces assets > 0*
noFreeAssets	Verified	*Round-trip withdraw(deposit(X)) <= X*
tellerDoesntHoldTokens	Verified	*Teller never holds tokens*

vaultCannotChange	Verified	*Teller.vault is immutable*
depositNonceNeverGoesDown	Verified	*Deposit nonce monotonically increases*
tellerPaused_valuesFrozen	Verified	*When paused, nonce frozen (except refundDeposit)*
tellerPaused_methodsRevert	Verified	*Public deposit/withdraw revert when paused*
dustFavorsTheHouse	Verified	*Rounding favors the vault*
noDynamicCalls	Verified	*No unauthorized external calls*
onlyContributionMethodsReduceAssets	Verified	*User assets decrease only via deposit methods*
withdrawingProducesAssets	Verified	*Burning shares > dust yields assets > 0*

Group 4: Math & Permissions

Status: Conditionally verified

Rule Name	Status	Description
yieldAccrualsMonotonic	Verified	*Yield accrual timestamp monotonic*
yieldIntegrity	Verified	*Realized yield ≤ pending gains; pending ≤ total pool*
streamingRateConsistency	Verified	*pendingGains <= vestingGains; past end time all gains pending*
accessControlYieldParams	Required a manual	*minVestingTime <= maxVestingTime*

	handler workaround	
deniedUsers_balanceNonDecreasing	Required a manual handler workaround	*Denied-from users can't lose shares*
deniedUsers_balanceNonIncreasing	Verified	*Denied-to users can't gain shares*
vaultSolvencyMulti	Verified	*Multi-asset solvency (skipped after fee claims/rate changes)*
vaultSolvency_1Asset	Verified	*Single-asset solvency (base asset only)*

Group 5: Math Conversion Properties (Both Systems)

Status: Verified

Rule Name	Status	Description
convertToAssetsWeakAdditivity	Verified	*convert(A) + convert(B) <= convert(A+B) + 1*
convertToSharesWeakAdditivity	Verified	*convert(A) + convert(B) <= convert(A+B) + 1*
conversionWeakMonotonicity	Verified	*A < B => convert(A) <= convert(B)*
zeroAllowanceOnAssets	Verified	*Users don't grant allowance to Teller*
conversionOfZero	Verified	*convert(0) == 0*

totalSupplyLEqCap	Verified	*After deposit: totalSupply <= depositCap*
weakAdditivityGeneral	Verified	*General additivity with dynamic amounts*

Group 6: Virtual Share Price (YS Only)

Status: Verified

Rule Name	Status	Description
virtualPriceUpperBound	Verified	`deposit(0) == 0` `withdraw(0) == 0`
virtualPriceLowerBound	Verified	`withdraw(deposit(X)) <= X`
pendingGainsRateRelationship	Verified	`A < B => withdraw(A) <= withdraw(B)`

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	2	-	-
Total	2	-	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
I-01	`setFirstDepositTimestamp()` + stale `vestingGains` creates invalid state	Informational	Acknowledged
I-02	`uint128` and `uint96` truncation causes silent precision loss and overflows	Informational	Acknowledged

Informational Issues

I-01. setFirstDepositTimestamp() + Stale vestingGains Creates Invalid State

Invariant 14: `startVestingTimeLEendVestingTime`
Status: Open (Handler Workaround Applied)

Description:
Two distinct factors create unsynchronized state:

1. `_updateExchangeRate()` doesn't clear vestingGains when `totalSupply == 0`
2. `setFirstDepositTimestamp()` only updates `startVestingTime`, not `endVestingTime`

Reproduction:

```
JavaScript
1. depositYS       → creates shares
2. vestYield       → sets vestingGains, startTime, endTime
3. warpTime  → time passes endTime
4. withdrawYS(all) → totalSupply = 0, but vestingGains NOT cleared
5. warpTime        → more time passes
6. bulkDepositYS   → setFirstDepositTimestamp() sets startTime = now
                     endTime remains old value → startTime > endTime
```

Recommendation:
If striving for absolute state consistency, consider resetting the end timestamp alongside the start timestamp when we deposit on an empty vault

Customer's response:
[Probably expected behavior, leaving as placeholder]

I-02. `uint128` and `uint96` truncation causes silent precision loss and overflows

Invariant 36: `invariant_virtualPriceUpperBound`
Invariant 18: `invariant_exchangeRatePostLoss`
Status: Open (Handler Workarounds Applied)

Description for 36:
When `lastVirtualSharePrice` becomes extremely large (typically after vesting large yields with very few shares), the `uint128` cast in `_calculateSharePriceFromVirtual()` silently truncates the value, causing `lastSharePrice` to be wildly different from the true converted virtual price.

Counter-example:

```
JavaScript
1. bulkDepositYS(...) → Creates initial shares
2. warpTime(huge) → Time warp (bounded to 365 days max)
3. bulkWithdrawYS(...) → Partial withdrawal (most shares gone, few remain)
4. updateVestingParams(...) → Configure vesting params
5. vestYield(8.369e46, ...) → Vest yield - large yield / few shares = huge price increase
6. warpTime(huge) → Another time warp
7. bulkWithdrawYS(...) → Final withdrawal
→ FAIL: convertedPrice (4.77e40) > lastSharePrice + 1 (9.74e37)
```

Description for 18:
Very low `totalSupply` (e.g., 1 share remaining after withdrawals) combined with vesting gains causes share price to grow astronomically large, which then triggers silent truncation at multiple points in the code, leading to the HWM deviating in the wrong direction.

Counter-example:

```
JavaScript
1. warpTime(3420632)          → ~39 days
2. bulkDepositYS(...)         → First deposit, creates vesting schedule
3. bulkWithdrawYS(...)        → Withdraw most shares (leaving 1 share!)
```

```
4. updateExchangeRateYS(...)  → Realize vesting gains
   → lastSharePrice = 2.74e36 (above uint96 max of 7.9e28)
   → rate = uint96(2.74e36) = 4.35e28 (truncated via modulo)
   → hwm = 4.35e28 (same truncated value)
5. postLoss(...)              → Post loss
   → lastSharePrice = 2.64e36 (DECREASED by ~3.6%)
   → rate = uint96(2.64e36) = 6.05e28 (DIFFERENT truncation!)
   → hwm = 4.35e28 (unchanged)
   → FAIL: rate (6.05e28) > hwm (4.35e28)
```

Recommendation:
Add overflow protection at truncation points if values are deemed production possible. Additionally, consider syncing HWM after any rate change, capping the maximum price increase to prevent overflow.

Customer's response:
[Probably expected behavior, leaving as placeholder]

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.